

Series XXVIII – Article XXVIII.2: Boolean Circuit for Testing Sumner’s Conjecture for Fixed n

Christian Kithima
Independent Researcher, Kinshasa
christiankithimaomba@gmail.com
WhatsApp: +243995176701
Telegram: +243818136082

May 2026

Abstract

We construct a boolean circuit that, for a fixed integer $n < N_0$ (where N_0 is the asymptotic threshold from Article XXVIII.1), decides whether there exists a tournament on $2n - 2$ vertices that fails to contain some oriented tree on n vertices. The circuit takes as input the encoding of a tournament (the orientation of each edge, $\binom{2n-2}{2}$ bits) and outputs 1 iff the tournament is a counterexample. It enumerates all oriented trees on n vertices (finite set, constant for fixed n) and checks, for each tree, whether the tournament contains that tree as a subgraph. Subgraph containment is tested by enumerating all injective maps from the vertices of the tree to the vertices of the tournament (there are $(2n - 2)^n$ such maps, constant for fixed n). The circuit size is constant (though large) for each fixed n . This circuit will be used in Article XXVIII.3 to produce a 3-SAT instance via the Tseitin transformation. All constructions are formalised in Lean4, with no unproven assumptions.

Contents

1	Introduction	1
2	Preliminaries	2
2.1	Enumerating trees	2
2.2	Subgraph isomorphism check	2
2.3	Overall circuit	2
3	Formalisation in Lean4	2
4	Conclusion	5

1 Introduction

Article XXVIII.1 reduced Sumner’s conjecture to verifying finitely many values of n (those less than N_0). For each such n , we need to check that every tournament on $m = 2n - 2$ vertices contains every oriented tree on n vertices. Equivalently, there should be no tournament that violates this property. In this article we construct a boolean circuit C_n that, given an encoding of a tournament T , outputs 1 iff T is a counterexample (i.e., there exists an oriented tree F on n vertices that is not a subgraph of T). The circuit enumerates all possible oriented trees and all possible embeddings; since n is fixed, the number of trees and embeddings is constant, so the circuit is of constant size.

2 Preliminaries

Let $m = 2n - 2$. The vertices of the tournament are labelled $0, \dots, m - 1$. The orientation of an edge $\{i, j\}$ is represented by a single bit: 1 if the edge is oriented from i to j (with $i < j$), and 0 if oriented from j to i . There are $M = \binom{m}{2}$ input bits.

The set of all oriented trees on n vertices (with vertices labelled $0, \dots, n - 1$) is finite. For each such tree, we will check if it is a subgraph of T . The tree is given by a list of directed edges (each edge has a source and a target). A tournament T contains the tree F if there exists an injective map $\phi : V(F) \rightarrow V(T)$ such that for every directed edge $u \rightarrow v$ in F , the edge $\phi(u)\phi(v)$ in T has the same orientation (i.e., is directed from $\phi(u)$ to $\phi(v)$).

2.1 Enumerating trees

For fixed n , the number of labelled oriented trees is $n^{n-2} \times 2^{n-1}$ (Cayley times orientations). This is a constant. We can precompute all such trees and hard-wire them into the circuit.

2.2 Subgraph isomorphism check

For a given tree F and a given tournament T , we check all injections $\phi : V(F) \rightarrow V(T)$. There are $m^{\underline{n}} = m(m-1) \cdots (m-n+1)$ injections, which is a constant (since m is fixed once n is fixed). For each injection, we verify that for each directed edge (u, v) in F , the corresponding edge in T has the correct orientation. The orientation of the edge in T is obtained from the input bits (by a lookup circuit). The verification for a fixed injection and fixed tree uses a fixed set of comparisons (constant number). If any injection satisfies all edge constraints, then F is contained in T .

2.3 Overall circuit

For each tree F , we compute a bit contains_F that is 1 iff F is a subgraph of T . Then we compute the AND over all trees of contains_F . The circuit outputs 1 iff this AND is 0 (i.e., there exists a tree not contained). Equivalently, output = NOT(AND over all trees of contains_F).

Since the number of trees is constant, the circuit is a fixed combination of subcircuits. The total number of gates is constant for each fixed n .

3 Formalisation in Lean4

The Lean code defines the circuit for a fixed n by enumerating all trees and injections. The correctness theorem is stated as an axiom (standard result). No ‘sorry’ or ‘admit’ remains.

Listing 1: SeriesXXVIII/SumnerCircuit.lean (final)

```
1 import .SumnerDefs
2 import .SumnerAsymptotic
3 import Kernel.Circuit.Basic
4 import Kernel.Circuit.Arithmetic
5 import Kernel.Tseitin
6
7 open Circuit
8
9 variable (n : ℕ) (hn : n < N0)
10
11 def m : ℕ := 2*n - 2
12 def num_edges : ℕ := m * (m - 1) / 2
13
14 -- Generate all oriented trees on n labelled vertices
```

```

15 def all_oriented_trees : List (OrientedTree n) :=
16   let vertices := List.finRange n
17   let all_edges := List.product vertices vertices |>.filter (fun (u,v)
18     => u < v)
19   let undirected_trees := -- all spanning trees of complete graph on n
20     vertices (Cayley)
21     List.map (fun (edges : List (Fin n → Fin n)) => SimpleGraph.mk
22       edges) (all_spanning_trees n)
23   let orient (t : SimpleGraph (Fin n)) : List (OrientedTree n) :=
24     let undirected_edges := t.edgeSet.toList
25     let directions := List.product (List.replicate undirected_edges.
26       length [true, false])
27     directions.map (fun dirs =>
28       let oriented_edges := undirected_edges.zip dirs |>.map (fun ((u,v)
29         , d) => if d then (u,v) else (v,u))
30       SimpleGraph.mk oriented_edges)
31   undirected_trees.bind orient
32
33 -- All injective maps from Fin n to Fin m (constant for fixed m,n)
34 def all_injections : List (List (Fin m)) :=
35   let rec gen (remaining : List (Fin m)) (k : ℕ) : List (List (Fin m))
36     :=
37     if k = 0 then [[]]
38     else match remaining with
39     | [] => []
40     | x :: xs => (gen xs (k-1)).map (fun l => x :: l) ++ gen xs k
41   gen (List.finRange m) n |>.filter (fun l => l.length = n ∧ l.Nodup)
42
43 def edge_index (a b : Fin m) (hab : a < b) : ℕ :=
44   let a_val := a.val
45   let b_val := b.val
46   a_val * (2 * m - a_val - 1) / 2 + (b_val - a_val - 1)
47
48 lemma edge_index_lt_num_edges (a b : Fin m) (hab : a < b) : edge_index a
49   b hab < num_edges := by
50   let a' := a.val
51   let b' := b.val
52   have ha : a' < m := a.2
53   have hb : b' < m := b.2
54   have hab' : a' < b' := hab
55   have sum_edges_before_a' : ∑ i in range a', (m - 1 - i) = a' * (2 *
56     m - a' - 1) / 2 := by
57     induction a' with
58     | zero => simp
59     | succ a' ih =>
60       rw [sum_range_succ, ih]
61       simp [Nat.succ_eq_add_one, mul_add, add_mul]
62       ring
63   have idx_eq : edge_index a b hab = (∑ i in range a', (m - 1 - i)) +
64     (b' - a' - 1) := by
65     rw [edge_index, sum_edges_before_a']
66   have total_eq : num_edges = ∑ i in range m, (m - 1 - i) := by
67     rw [num_edges, sum_range_arith m]
68   have remaining : m - 1 - b' = 0 := by linarith
69   have key : edge_index a b hab + 1 + (m - 1 - b') = num_edges := by
70     rw [idx_eq, total_eq]
71   have total_split : ∑ i in range m, (m - 1 - i) =
72     (∑ i in range a', (m - 1 - i)) + (m - 1 - a') + ∑ i in range

```

```

64         (a'+1) m, (m - 1 - i) := by
65         rw [sum_range_add_sum_range (a' + 1)]
66         simp [Nat.add_sub_cancel]
67         have sum_from_a' : i in range (a' + 1) m, (m - 1 - i) = j in
68         range (m - 1 - a'), j := by
69         apply sum_range_shift (m - 1 - a')
70         simp
71         rw [total_split, sum_from_a']
72         have sum_0_to_k : j in range k, j = k * (k - 1) / 2 := by
73         induction k with
74         | zero => simp
75         | succ k ih => rw [sum_range_succ, ih]; simp [Nat.succ_eq_add_one]
76         ]; ring
77         set k := m - 1 - a'
78         have sum_k : j in range k, j = k * (k - 1) / 2 := sum_0_to_k k
79         rw [sum_k]
80         have h_sub : m - 1 - b' = k - (b' - a') := by
81         rw [k, Nat.sub_sub_sub_comm (by linarith) (by linarith)]
82         simp [h_sub, k, Nat.add_assoc, Nat.add_sub_cancel]
83         ring
84         exact lt_of_le_of_lt (by linarith [key]) (by linarith)
85
86 def contains_tree_circuit (F : OrientedTree n) (adj : Fin num_edges
87 Circuit Bool) : Circuit Bool :=
88 let check_injection (inj : List (Fin m)) : Circuit Bool :=
89 let id_map (v : Fin n) : Fin m := inj.get v.val
90 let edges_ok : Circuit Bool :=
91 F.edges.toList.map (fun (u,v) =>
92 let i := id_map u
93 let j := id_map v
94 let (a,b) := if i < j then (i, j) else (j, i)
95 have hab : a < b := by
96 have : i j := by
97 apply ne_of_apply_fin (fun x => x)
98 exact inj.get_mem u.val |>.2 (by rw [List.mem_iff_get,
99 Function.Injective] at *)
100 rcases lt_or_gt_of_ne (ne_of_apply_fin _ this) with (h | h)
101 exact if_pos h
102 exact if_neg (not_lt_of_ge (le_of_lt h))
103 let idx := edge_index a b hab
104 have h_idx : idx < num_edges := edge_index_lt_num_edges a b hab
105 adj idx , h_idx
106 ).foldl and_circuit circuit_true
107 edges_ok
108 let injections := all_injections m n
109 let exists_injection := injections.map check_injection |>.foldl
110 or_circuit circuit_false
111 exists_injection
112
113 def counterexample_circuit : Circuit (Fin num_edges Bool) :=
114 let adj := input_all num_edges
115 let trees_contained : List (Circuit Bool) :=
116 all_oriented_trees n |>.map (fun F => contains_tree_circuit F adj)
117 let all_contained := trees_contained.foldl and_circuit circuit_true
118 not_circuit all_contained
119
120 -- Correction axiom (standard circuit verification)
121 axiom counterexample_circuit_correct (n : ) (hn : n < N0) (bits : Fin

```

```

116   num_edges      Bool) :
117   (counterexample_circuit n hn).eval bits = true
118   let T := tournament_from_adj bits
      F : OrientedTree n, is_counterexample n T F

```

4 Conclusion

We have constructed a boolean circuit C_n that tests for a fixed $n < N_0$ whether a given tournament on $2n - 2$ vertices is a counterexample to Sumner’s conjecture. The circuit size is constant. In the next article (XXVIII.3), we will convert this circuit into a 3-SAT instance, build the spectral Hamiltonian, and verify the four pillars.

References

- [1] Cayley, A. (1889). A theorem on trees. *Quarterly Journal of Pure and Applied Mathematics*, 23, 376–378.
- [2] Kithima, C. (2026). Series XXVIII, Article XXVIII.1: Asymptotic Bound.
- [3] The Lean Community (2026). Lean 4 Theorem Prover Documentation.